

Τεχνική Αναφορά

Scikit-learn Agentic RAG Agent

Εφαρμογές Επιστήμης Δεδομένων και Τεχνητής Νοημοσύνης - CSIS-E5

Ατομική εργασία - Ιούνιος 2026

Η παρούσα αναφορά συνοψίζει την τελική υλοποίηση ενός Agentic Retrieval-Augmented Generation (RAG) συστήματος για ερωτήσεις πάνω στο επίσημο scikit-learn documentation. Η έκδοση αυτή είναι συμπυκνωμένη για παράδοση και αφαιρεί τα περιττά browser headers/footers της προηγούμενης PDF εξαγωγής.

Περιεχόμενα

- 1. Εισαγωγή και στόχος συστήματος
- 2. Corpus και θεματικό πεδίο
- 3. Αρχιτεκτονική συστήματος
- 4. Ingestion, chunking και metadata filtering
- 5. RAG pipeline και grounded generation
- 6. Agent architecture
- 7. Evaluation methodology και αποτελέσματα
- 8. HAIC benchmarking
- 9. Παρατηρησιμότητα με Langfuse
- 10. Production setup / Docker
- 11. Limitations και επόμενα βήματα
- 12. Δήλωση συνεισφοράς

1. Εισαγωγή και στόχος συστήματος

Η εργασία αφορά την ανάπτυξη ενός Agentic RAG συστήματος που απαντά σε ερωτήματα σχετικά με supervised classification workflows στο scikit-learn. Το σύστημα δεν βασίζεται αποκλειστικά στις γενικές γνώσεις ενός LLM. Αντίθετα, ανακτά σχετικά αποσπάσματα από επιλεγμένο corpus, τα χρησιμοποιεί ως context και παράγει grounded απαντήσεις με πηγές.

Ο βασικός στόχος ήταν η δημιουργία ενός ολοκληρωμένου pipeline που καλύπτει corpus ingestion, chunking, embeddings, Qdrant vector store, retrieval, grounded generation, agent routing, evaluation, HAIC benchmarking, Langfuse observability και αναπαραγωγή εκτέλεση μέσω Docker.

- Το σύστημα απευθύνεται σε αρχάριους και μεσαίου επιπέδου χρήστες Python/ML.
- Υποστηρίζει ερωτήσεις για preprocessing, pipelines, train/test split, cross-validation, metrics, Logistic Regression και RandomForestClassifier.
- Η απάντηση βασίζεται στο retrieved context και όχι σε ανεξέλεγκτη παραγωγή από το LLM.
- Αν η πληροφορία δεν υπάρχει στο corpus, το σύστημα πρέπει να το δηλώνει αντί να επινοεί απάντηση.

2. Corpus και θεματικό πεδίο

Το corpus αποτελείται από 10 curated σελίδες του επίσημου scikit-learn documentation. Το θεματικό πεδίο περιορίστηκε σε supervised classification workflows, ώστε το corpus να είναι μικρό αλλά ουσιαστικό και επαρκές για αξιολόγηση διαφορετικών ερωτημάτων.

ID	Θέμα	Ρόλος στο corpus
D01	Εισαγωγή στο scikit-learn	Βασικές έννοιες: fitting, predicting, transformers, pipelines, evaluation.
D02	Preprocessing / scaling	StandardScaler και επίδραση του scaling σε αλγορίθμους.
D03	Pipelines	Οργάνωση ML workflows και μείωση data leakage.
D04	Train/test split	Διαχωρισμός δεδομένων σε train και test sets.
D05	Cross-validation	Αξιολόγηση με folds και cross-validation strategy.
D06	Hyperparameter tuning	GridSearchCV, RandomizedSearchCV και parameter selection.
D07	Metrics	Accuracy, precision, recall, F1-score και scoring.
D08	Logistic Regression	Classifier και βασικές παράμετροι regularization.
D09	Random Forests	Εννοιολογική περιγραφή ensemble methods και random forests.
D10	RandomForestClassifier	API page και παράμετροι όπως max_depth, min_samples_split, max_features, ccp_alpha.

Το corpus υποστηρίζει conceptual questions, procedural/how-to questions και decision-support questions. Παραδείγματα είναι ερωτήσεις για το πότε χρησιμοποιείται feature scaling, πώς δημιουργείται pipeline με StandardScaler και LogisticRegression, ποιο metric είναι κατάλληλο για imbalanced classification και ποιες παράμετροι του RandomForestClassifier ελέγχουν την πολυπλοκότητα του μοντέλου.

3. Αρχιτεκτονική συστήματος

Η συνολική ροή ξεκινά από το documentation corpus, συνεχίζει με δύο στρατηγικές chunking, δημιουργεί embeddings, τα αποθηκεύει στο Qdrant και εκτελεί retrieval για να δοθεί context στο LLM. Πάνω από το RAG pipeline υπάρχει agent layer που επιλέγει retrieval tool ανάλογα με την ερώτηση.

Corpus -> Ingestion -> Chunking (v1_fixed / v2_structured) -> Embeddings -> Qdrant -> Retrieval -> Context -> Grounded answer -> Agent layer -> Evaluation / Langfuse / Docker

Component	Υλοποίηση / Ρόλος
Corpus ingestion	Συλλογή και τοπική αποθήκευση scikit-learn documentation pages.
Chunking	Σύγκριση fixed-size και structured/section-based chunking.
Embeddings	text-embedding-3-small για chunks και ερωτήσεις.
Vector store	Qdrant collections: sklearn_rag_v1_fixed και sklearn_rag_v2_structured.
RAG pipeline	Ανάκτηση top-k chunks, context construction, grounded generation με gpt-4.1-mini.
Agent layer	Rule-based baseline και LangGraph ReAct agent με δύο retrieval tools.
Evaluation	Golden test set 30 ερωτήσεων, retrieval metrics, LLM-as-judge, Tool Call Accuracy, HAIC.
Observability / Production	Langfuse traces για representative runs και Docker Compose με Qdrant + Python demo service.

4. Ingestion, chunking και metadata filtering

Για το ingestion χρησιμοποιήθηκε το script `src/ingestion/download_corpus.py`, το οποίο αποθηκεύει καθαρισμένα κείμενα και metadata. Στη συνέχεια υλοποιήθηκαν δύο chunking strategies ώστε να γίνει ποσοτική σύγκριση της ποιότητας retrieval.

Strategy	Περιγραφή	Output	Chunks
v1_fixed	Fixed-size chunking με overlap. Απλό baseline.	data/processed/ v1_fixed_chunks.jsonl	402
v2_structured	Section-based chunking με βάση headings/sections. Διατηρεί καλύτερα το νοηματικό πλαίσιο.	data/processed/ v2_structured_chunks.jsonl	375

Κάθε chunk συνοδεύεται από metadata: `chunk_id`, `doc_id`, `title`, `topic`, `url`, `section`, `heading_level`, `chunking_strategy` και `text`. Το metadata schema επιτρέπει εμφάνιση πηγών και metadata-filtered retrieval μέσω Qdrant payload filtering.

Παράδειγμα: για την ερώτηση "Which RandomForestClassifier parameters can control model complexity?" το topic filter `random_forest_classifier` περιορίζει την αναζήτηση στα chunks της D10 API πηγής. Αυτό είναι χρήσιμο επειδή απλή semantic similarity μπορεί να φέρει γενικότερα chunks για Random Forests αντί για την ακριβή API σελίδα.

5. RAG pipeline και grounded generation

Το βασικό RAG pipeline υλοποιείται στο `src/rag/rag_answer.py`. Η ερώτηση γίνεται `embedding`, εκτελείται `vector search` στο Qdrant, ανακτώνται τα `top-k chunks`, χτίζεται `context` και το LLM παράγει απάντηση βασισμένη στο `context`.

question -> embedding -> Qdrant vector search -> retrieved chunks -> context construction -> grounded LLM answer -> answer + sources

Script	Ρόλος
<code>src/rag/rag_answer.py</code>	Single-question RAG απάντηση.
<code>src/rag/test_retrieval.py</code>	Έλεγχος απλής semantic retrieval.
<code>src/rag/test_filtered_retrieval.py</code>	Έλεγχος metadata-filtered retrieval.
<code>src/rag/run_golden_rag.py</code>	Εκτέλεση RAG στο golden test set.
<code>src/rag/llm_judge_evaluation.py</code>	LLM-as-judge evaluation των απαντήσεων.

Το prompt δίνει ρητή οδηγία για grounding: αν η απάντηση δεν υποστηρίζεται από το `context`, το σύστημα πρέπει να δηλώσει ότι δεν υπάρχουν αρκετές πληροφορίες. Αυτό ελέγχθηκε με `out-of-context` ερώτηση για το FIFA World Cup 2022, όπου το σύστημα απάντησε ότι το διαθέσιμο `context` δεν επαρκεί.

6. Agent architecture

Πάνω από το RAG pipeline υλοποιήθηκε agent layer για επιλογή κατάλληλου retrieval tool. Υπάρχουν δύο εκδοχές: rule-based routing agent ως baseline/fallback και LangGraph ReAct agent ως πιο agentic υλοποίηση.

Tool	Arguments	Ρόλος
rag_retriever	question, top_k	Γενική αναζήτηση στο συνολικό corpus χωρίς metadata filter.
metadata_filtered_retriever	question, topic_filter, top_k	Αναζήτηση με topic filter για API-specific ή θεματικά εστιασμένες ερωτήσεις.

Χαρακτηριστικό	Rule-based agent	LangGraph ReAct agent
Routing	Deterministic keyword rules	LLM-based tool selection
Tools	2 retrieval tools	2 retrieval tools
Metadata filtering	Ναι	Ναι
ReAct λογική	Περιορισμένη / conceptual	Ναι, μέσω LangGraph
Σταθερότητα	Υψηλή	Εξαρτάται από LLM routing
Ρόλος	Baseline και fallback	Improved agentic version

Στο demo με RandomForestClassifier, ο agent επιλέγει metadata_filtered_retriever με topic_filter=random_forest_classifier και top_k=5. Αυτό σημαίνει ότι ανακτώνται τα πέντε πιο σχετικά chunks μόνο από το συγκεκριμένο topic, αντί να γίνει αναζήτηση σε όλο το corpus.

7. Evaluation methodology και αποτελέσματα

Η αξιολόγηση έγινε με golden test set 30 ερωτήσεων, αποθηκευμένο στο evaluation/golden_test_set_v0-3.xlsx. Οι ερωτήσεις καλύπτουν conceptual, procedural και decision-support περιπτώσεις. Η αξιολόγηση περιλαμβάνει retrieval metrics, σύγκριση v1/v2, LLM-as-judge metrics, Tool Call Accuracy και HAIC benchmarking.

Evaluation layer	Μετρικές / Έλεγχος
Retrieval evaluation	Expected source in top-k και Expected source top-1.
v1 vs v2 comparison	Σύγκριση των Qdrant collections sklearn_rag_v1_fixed και sklearn_rag_v2_structured.
LLM-as-judge	Answer Relevancy, Faithfulness, Context Precision, Context Recall, Expected Answer Alignment.
Tool Call Accuracy	Αν ο agent επέλεξε αναμενόμενο tool και σωστό topic filter.
Failure analysis	Ανάλυση περιπτώσεων όπου metadata filtering βελτιώνει το context.

Metric	v1_fixed	v2_structured	Ερμηνεία
Expected source in top-k	93.33%	96.67%	Το σωστό source εμφανίζεται συχνότερα στα retrieved chunks.
Expected source top-1	73.33%	80.00%	Το σωστό source εμφανίζεται συχνότερα πρώτο.

Η structured στρατηγική v2_structured είχε καλύτερη retrieval επίδοση. Παρήγαγε λιγότερα αλλά πιο συνεκτικά chunks, πιο κοντά στις πραγματικές ενότητες του documentation. Η LLM-as-judge αξιολόγηση έδειξε υψηλές επιδόσεις περίπου στο εύρος 4.73-4.97/5 στις βασικές διαστάσεις. Η Tool Call Accuracy του rule-based baseline ήταν 100% στις 30 ερωτήσεις, κάτι που δείχνει συνέπεια των deterministic κανόνων αλλά όχι απαραίτητα πλήρη γενίκευση σε άγνωστες ερωτήσεις.

8. HAIC benchmarking

Για τη Human-AI Collaboration διάσταση εφαρμόστηκε professor-style HAIC artifact με schema haic.decisions_artifact.v1. Το benchmark έτρεξε πάνω στις 30 ερωτήσεις του golden test set και κατέγραψε 150 events: human query, AI routing, tool call, AI response και accept/reject proxy.

Metric	Value	Ερμηνεία
EL	0.000	Effort loss compared to baseline.
Tr	0.967	Fraction of accepted AI responses.
HCL	0.883	Υψηλότερη τιμή δείχνει χαμηλότερο cognitive load.
F	51.404	Interaction events per minute.
A	0.197	Adaptability across early vs late decisions.
D	1.167 s	Mean duration per decision event.
EfficiencyScore	1.000	Composite efficiency score.

Τα diagnostics κατέταξαν το σύστημα ως Efficient & Trusted και Smooth collaboration. Ο σημαντικός περιορισμός είναι ότι δεν έγινε πραγματικό user study: το accept/reject υπολογίστηκε offline με proxy κριτήριο, δηλαδή η απάντηση θεωρείται accepted όταν η αναμενόμενη πηγή εντοπίζεται στα retrieved chunks. Επομένως, οι HAIC μετρικές είναι offline approximation και όχι πραγματική ανθρώπινη αξιολόγηση.

9. Παρατηρησιμότητα με Langfuse

Για observability χρησιμοποιήθηκε Langfuse tracing σε representative RAG και agent runs. Η υλοποίηση βασίζεται στο helper `src/observability/langfuse_tracing.py`. Το helper ενεργοποιεί Langfuse μόνο όταν υπάρχουν `LANGFUSE_PUBLIC_KEY`, `LANGFUSE_SECRET_KEY` και `LANGFUSE_HOST`. Αν λείπουν, τα scripts συνεχίζουν κανονικά χωρίς να αποτύχουν.

Traced entry point	Τι καταγράφεται
<code>src/rag/rag_answer.py</code>	Ερώτηση χρήστη, retrieval call, retrieved chunk metadata και grounded answer generation.
<code>src/agent/langgraph_agent.py</code>	Agent question, tool selection, tool arguments, retrieved chunks και τελική απάντηση.
<code>src/agent/traced_agent_demo.py</code>	Representative traced demo για παρουσίαση.

Το tracing δεν καλύπτει κάθε batch evaluation script από προεπιλογή, ώστε να μην αυξάνει άσκοπα κόστος και χρόνο εκτέλεσης. Καλύπτει όμως representative RAG/agent runs που είναι κατάλληλα για παρουσίαση, debugging και ανάδειξη tool selection.

10. Production setup / Docker

Η τελική υλοποίηση περιλαμβάνει Docker Compose setup με δύο services: Qdrant vector store και Python app/demo service. Το app service χρησιμοποιεί .env μέσω env_file, συνδέεται στο Qdrant με QDRANT_HOST=qdrant και ελέγχει αν υπάρχει η collection sklearn_rag_v2_structured. Αν λείπει ή είναι άδεια, την ξαναδημιουργεί από το data/processed/v2_structured_chunks.jsonl.

```
docker compose up --build
```

Το Docker demo εκτελεί τρεις representative ερωτήσεις και αποθηκεύει το αποτέλεσμα στο outputs/docker_demo_run.md. Οι τρεις ερωτήσεις δείχνουν: γενικό RAG retrieval, metadata-filtered retrieval για RandomForestClassifier και out-of-context συμπεριφορά όπου το σύστημα δηλώνει ότι το context δεν επαρκεί.

Demo ερώτηση	Αναμενόμενη συμπεριφορά
Classification workflow	Χρήση rag_retriever και αναζήτηση στο πλήρες corpus.
RandomForestClassifier parameters	Χρήση metadata_filtered_retriever με topic_filter=random_forest_classifier.
FIFA World Cup 2022	Out-of-context απάντηση: δεν υπάρχουν αρκετές πληροφορίες στο διαθέσιμο context.

11. Limitations και επόμενα βήματα

Παρότι το σύστημα καλύπτει τις βασικές απαιτήσεις της εργασίας, υπάρχουν περιορισμοί που πρέπει να ληφθούν υπόψη.

Περιορισμός	Ερμηνεία
Περιορισμένο corpus	Χρησιμοποιούνται 10 curated scikit-learn pages, όχι ολόκληρο το documentation.
Rule-based baseline	Ο αρχικός router είναι σταθερός αλλά όχι πλήρως agentic.
LangGraph routing dependence	Το LLM-based tool selection μπορεί θεωρητικά να επιλέξει λάθος tool. Υπάρχει hybrid/fallback λογική.
Offline HAIC evaluation	Δεν έγινε πραγματικό user study. Το accept/reject υπολογίστηκε offline με proxy.
Representative Langfuse tracing	Καλύπτει representative RAG/agent runs, όχι όλα τα batch evaluation scripts.
Demo-oriented Dockerization	Υπάρχει Qdrant + Python demo service, αλλά όχι πλήρες interactive UI/API.

Επόμενα βήματα θα ήταν η επέκταση του corpus σε περισσότερες ενότητες του scikit-learn, πραγματικό user study για HAIC, πλήρες interactive UI/API πάνω από τον RAG agent, επέκταση Langfuse tracing σε όλα τα batch evaluation scripts, ξεχωριστή αξιολόγηση του LangGraph routing σε μεγαλύτερο test set και προσθήκη hybrid search ή reranking.

12. Δήλωση συνεισφοράς

Η εργασία υλοποιήθηκε ως ατομικό project. Η ανάπτυξη περιλαμβάνει επιλογή corpus, ingestion, chunking, indexing στο Qdrant, RAG pipeline, rule-based agent baseline, LangGraph ReAct agent, evaluation, HAIC benchmarking, Langfuse tracing, Docker setup, README και τεχνική τεκμηρίωση.

- Δύο ingestion/chunking strategies και δύο Qdrant collections.
- Βασικό RAG pipeline με grounded generation και out-of-context handling.
- Δύο retrieval tools και agentic routing με metadata filtering.
- Golden test set 30 ερωτήσεων και αξιολόγηση retrieval/LLM-as-judge/tool calls.
- HAIC benchmarking, Langfuse tracing και Docker Compose setup για Qdrant + Python demo service.
- Τελικό README, τεχνική αναφορά και presentation slides.

Συνολικά, το project υλοποιεί ένα λειτουργικό Agentic RAG σύστημα για το scikit-learn documentation, με έμφαση στην τεκμηριωμένη παραγωγή απαντήσεων, την αξιολόγηση, την παρατηρησιμότητα και την αναπαραγωγικότητα.